

01 개발자 가이드

CPF 공개 API 와 공통 기능을 선택해 Online 업무를 구현·검증하는 실무 매뉴얼

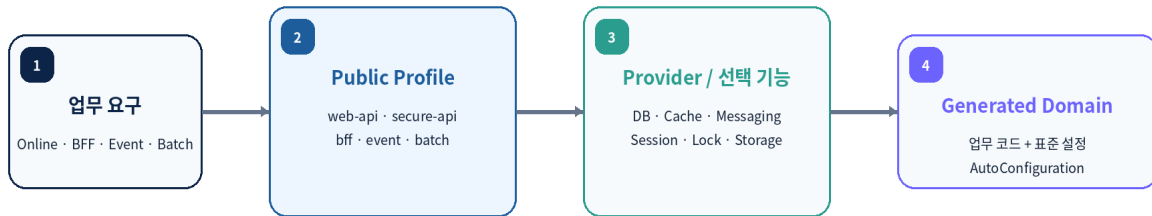
기준 Source: master bd5fcd7707cfbba366071e7e4556c5cbb9b974bc · 2026-08-15

1. 표준 개발 구조

CPF 업무 코드는 Framework 공통 → Domain 공통 → 업무 구현의 3 단 구조를 기본으로 합니다. Controller 는 HTTP 경계, Service 는 업무 규칙과 트랜잭션, Repository/DAO 는 데이터 접근을 담당하며 Domain Base 에는 실제로 반복되는 정책만 둡니다.

CPF 기능 선택 구조

업무 요구에 맞는 Public Profile 을 먼저 선택하고, 필요한 Provider·선택 기능만 추가



대표 선택 예

일반 REST API	web-api	DB 사용 시 JDBC/MyBatis/JPA 추가
보호 API	secure-api	Resource Server·Service Identity 정책
화면 Backend	bff	필요 시 Session Provider 추가
비동기 Event	event	Kafka/RabbitMQ/JMS/IBM MQ 중 선택
정기·대량 처리	batch	DB·Lock·Messaging 등 필요한 기능 조합

외부 연계 HTTP/TCP/SOAP·전문 등은 공개 연계 API/Profile 내부 구성으로 사용하며 Internal Leaf를 직접 조립하지 않습니다.

기능을 먼저 비교한 뒤 상세 구현과 실제 Education/Sample 로 내려가는 개발 문서 흐름

계층	주요 책임	CPF 기본 제공	업무 코드에서 할 일
Controller	요청/응답, 입력 검증, HTTP 경계	Context, Validation, 표준 응답 Helper	DTO 검증 후 Service 호출
Domain Base Controller	Domain 공통 Web 정책	Framework Base 기능 재사용	페이지 제한, Domain 공통 규칙
Service	업무 규칙, 트랜잭션, Domain/외부 호출	기본 문자열 검증, Context 연계	원자성·호출 결과·업무 오류 처리
Domain Base Service	Domain 공통 업무 정책	TransactionId/공통 정규화 확장	Domain 식별자, 공통 업무 Helper
Repository / DAO	DB 접근 계약	CRUD/Search/Bulk/Lock, Context Helper	Query·Mapper·Provider 연결
Provider	JDBC/MyBatis/JPA 실행	공개 Provider 와 표준 Paging/오류	필요 시 Provider 고유 기능 사용

2. Controller 기본 제공 기능

`CpfBaseController` 는 Controller 에서 반복되는 Context 조회, 입력 검증, 표준 응답 조립을 제공합니다. Business Controller 가 Spring/CPF 내부 구현을 다시 만들지 않도록 Domain Base 가 이 기능을 그대로 재사용합니다.

메서드	반환/동작	특징	주 사용 위치
<code>`requireCurrentContext()`</code>	현재 <code>`CpfContext`</code>	TransactionId, actor, tenant 등 현재 실행 Context 접근	업무 식별자/권한 보조 정보가 필요할 때
<code>`requireContext()`</code>	<code>`CpfContextSnapshot`</code>	현재 실행 현재 실행 Context 정보	비동기 전달/진단용 Context 가 필요할 때
<code>`requireText(value, field)`</code>	trim 된 문자열	null/blank 거부	Path/Query/업무 키 필수 검증
<code>`requireValue(value, field)`</code>	검증된 객체	null 거부	URI/객체 필수 검증
<code>`requireRule(condition, message)`</code>	조건 위반 시 검증 오류	간단한 공통 조건 검증	페이지 크기, 값 범위 등
<code>`ok(body)`</code>	HTTP 200	표준 <code>`ResponseEntity`</code> 생성	정상 조회/처리 결과
<code>`created(location, body)`</code>	HTTP 201	Location 검증 후 생성 응답	REST 등록 API
<code>`noContent()`</code>	HTTP 204	본문 없는 정상 응답	삭제/상태변경 등
<code>`executionFacts(operation)`</code>	비민감 실행 식별자 묶음	transactionId/executionId/actorId/tenantId 포함	Log/Audit/진단 연결

Domain Base 예시 ``MemberBaseController``는 ``normalizePageSize()``를 추가해 기본 20 건, 최대 200 건 정책을 Domain 공통으로 적용합니다. 이런 실제 반복 정책이 없으면 빈 Base Class 를 만들지 않습니다.

```
@RestController
@RequestMapping("/api/members")
public class MemberController extends MemberBaseController {
    @GetMapping("/{memberId}")
    public ResponseEntity<MemberResponse> detail(@PathVariable String memberId) {
        String id = requireText(memberId, "memberId");
        return ok(service.getMember(id));
    }
}
```

3. Service 기본 제공 기능과 Domain 공통 기능

``CpfBaseService``는 Runtime 이나 DB Provider 에 종속되지 않는 최소 검증 기능만 제공합니다. Domain Base Service 는 현재 Domain 에서 실제로 반복되는 TransactionId, 업무 키, 사용자 식별자 정책 등을 추가합니다.

구분	메서드/값	특징	사용 기준
Framework Base	<code>`requireText(value, field)`</code>	null/blank 거부 + trim	대부분의 업무 문자열 필수값
Member/External Domain Base	<code>`requireTransactionId()`</code>	현재 Context 의 TransactionId 검증	거래 식별자가 업무/외부 호출에 필요할 때
Member/External Domain Base	<code>`transactionSequence()`</code>	현재 거래 실행 순번	재시도/후속 처리 식별에 활용
Member/External Domain Base	<code>`actorId()`</code>	현재 operator 가 없으면 SYSTEM_CODE 사용	감사성 수정자 식별
Education Domain Base	<code>`normalizeEducationKey()`</code>	trim + 대문자 + 64 자 제한	Domain 공통 키 규칙 예시

설계 원칙 Service Base 에 Repository, HTTP Client, Batch Runtime 같은 특정 기술 책임을 넣지 않습니다. 공통 기능은 해당 Owner Starter/API 를 주입받아 사용하고, Domain Base 에는 Domain 공통 정책만 둡니다.

4. DAO/Repository 기본 제공 기능

4.1 `CpfBaseDao` 공통 메서드

메서드	동작	특징	사용 예
`requireCurrentContext()`	현재 Context 반환	DB 접근 시 거래/사용자 식별	Audit Column, Query Hint 보조
`requireContext()`	현재 실행 Context 정보 반환	비민감 실행 정보 보존	Adapter/비동기 경계
`requireText()`	문자열 필수 검증	공통 Validation 사용	Mapper namespace, 키 검증
`requireRule()`	조건 검증	공통 Validation 사용	Paging/범위 Guard
`statementId(namespace, statement)`	`namespace.statement` 생성	빈 namespace/statement 거부	MyBatis Statement ID
`boundedSize(requested, default, max)`	대량/페이지 크기 제한	0 이하 기본값, max 초과 제한	조회·Bulk 범위 제한
`executionFacts(operation)`	DB 실행 식별자 묶음	transactionId/executionId/actorId/tenantId	DB Audit/진단

4.2 공통 Repository 계약

기능	공개 API	특징	선택 기준
등록	`CpfCrudRepositoryPort.create()`	Entity/Aggregate 1 건 생성	일반 CRUD
단건 조회	`findById()`	`Optional<T>`	PK/고유키 상세
수정	`update()`	갱신 결과 Entity 반환	일반 수정
삭제	`deleteById()`	boolean 결과	물리/Adapter 정책에 따른 삭제
존재 확인	`existsById()`	본문 미조회 가능	중복·사전 검증
건수	`count()`	전체 건수	단순 건수
Page	`CpfSearchRepositoryPort.page()`	COUNT + page/size	전체 페이지 수가 필요한 화면
Slice	`slice()`	COUNT 없이 다음 여부	전체 건수가 불필요한 목록
Cursor	`cursor()`	Keyset/Cursor 기반	대량·연속 조회
대량 등록/수정/삭제	`CpfBulkRepositoryPort.*All()`	제한형 대량 변경	명시적으로 Bulk 가 필요한 경우
잠금 조회	`CpfLockingRepositoryPort.findById(id, lockMode)`	낙관/비관 잠금 선택	실제 불변조건 보호
Provider 고유 기능	`nativeRepository()`	고급 기능 연결 지점	공통 계약으로 표현하기 어려운 경우에만

4.3 `CpfDataOperations` / `CpfJdbcOperations`

API	메서드	용도	주의
`CpfDataOperations`	`selectList/selectOne/insert/update/delete`	Provider 와 독립적인 정의 작업 ID 기반 실행	업무 Service 보다 Generated Adapter/Infrastructure 에서 사용

API	메서드	용도	주의
`CpfDataOperations`	`inRollbackOnlyTransaction(callback)`	작업을 실행하되 항상 rollback-only	검증/Probe 처럼 DB 반영을 남기지 않을 때
`CpfJdbcOperations`	`queryList/queryOne/update`	JDBC Adapter 의 명시 SQL 실행	SQL 은 Vendor Catalog/정보에서 공급
`CpfJdbcOperations`	`inRollbackOnlyTransaction(callback)`	JDBC 실행 후 rollback-only	실제 반영 없는 DB 검증

5. DB 조회와 Paging 선택

DB 거래 개발 흐름

조회 방법을 먼저 고르고 Provider와 Vendor 선택 위치를 분리해서 이해



DB 거래 구현과 공식 Vendor 자산 확인 흐름

처리	권장 기능	핵심 특징	적합한 경우
단건	`findBy` / `selectOne` / `queryOne`	0~1 건	상세/PK 조회
존재	`existsBy`	본문 없이 존재 확인	중복/사전 검증
건수	`count`	건수만 반환	통계/전체 건수
Page	`CpfPageRequest` + `CpfPage`	COUNT 포함	전체 페이지 수 필요
Slice	`CpfSlice.fromLookAhead()`	size+1 조회로 `hasNext` 판단	전체 건수 불필요
Cursor	`CpfCursorPage`	정렬키 기반 연속 조회	대량/무한 스크롤
대량 변경	`CpfBulkRepositoryPort`	호출 범위를 제한한 Bulk	명시적 대량 변경
잠금	`CpfLockMode`	OPTIMISTIC/PESSIMISTIC 등	동시 수정 불변조건 보호

Paging 계약	주요 API	제약/특징
`CpfPageRequest`	`of(page,size)` , `offset()`	0-base, 기본 20, 최대 200
`CpfPage`	`of(content,request,total)` , `totalPages()` , `hasNext()`	COUNT 가 필요한 표준 Page

Paging 계약	주요 API	제약/특징
`CpfSlice`	`fromLookAhead(rows,page,size)`	size+1 행으로 다음 데이터 판단
`CpfCursorPage`	`last(items)` 및 cursor Adapter	`hasNext=true` 이면 nextCursor 필수

6. 트랜잭션 - 반드시 먼저 선택할 기준

DB 업무를 구현할 때는 메서드에 Annotation 을 붙이는 것보다 먼저 “어디까지 같이 성공/실패해야 하는가”를 결정합니다. CPF 의 Canonical Transaction Annotation 은 `@CpfTx`이며 Spring `Propagation` 과 `Isolation` 을 그대로 사용합니다.

6.1 `@CpfTx` 전체 옵션

옵션	기본값	역할	사용 기준
`id`	필수	거래 식별 ID	운영/Trace 에서 구분 가능한 안정 ID
`name`	필수	거래 명칭	업무 의미를 설명하는 명칭
`ownerDomain`	필수	소유 Domain	MBR/EXS 등 Domain 코드
`transactionManager`	빈 값	사용할 TransactionManager Bean	Manager 가 여러 개면 명시 필수
`propagation`	`REQUIRED`	기존 Tx 참여/분리 방식	REQUIRES_NEW 등 경계 선택
`isolation`	`DEFAULT`	DB Isolation 수준	특별한 동시성 요구가 있을 때만 변경
`readOnly`	`false`	조회 의도	순수 조회 업무
`timeoutSeconds`	`-1`	트랜잭션 제한 시간	장기 Lock 방지/업무 SLA
`requiredPermission`	빈 값	필요 권한 메타데이터	보안 정책 연계 시
`auditReasonRequired`	`false`	감사 사유 요구	위험/감사성 작업
`visibility`	`PUBLIC`	거래 공개 범위 메타데이터	공개/내부 정책
`gatewayAllowed`	`true`	Gateway 진입 허용 메타데이터	외부 진입 정책
`directAllowed`	`false`	Direct 진입 허용 메타데이터	직접 호출이 필요한 특수 경우

6.2 트랜잭션 전파 방식 선택표

전파 방식	현재 Tx 가 있을 때	현재 Tx 가 없을 때	대표 용도
`REQUIRED`	현재 Tx 에 참여	새 Tx 시작	일반 등록/수정/삭제 - 기본값
`REQUIRES_NEW`	현재 Tx 를 중단하고 새 Tx	새 Tx 시작	감사/이력/독립 후처리처럼 주 Tx 와 분리
`SUPPORTS`	현재 Tx 에 참여	Tx 없이 실행	조회 Helper 등 Tx 강제가 필요 없는 경우
`MANDATORY`	현재 Tx 에 참여	오류	상위 업무 Tx 안에서만 실행되어야 하는 내부 작업
`NOT_SUPPORTED`	현재 Tx 를 중단	Tx 없이 실행	DB Tx 밖에서 실행할 작업
`NEVER`	오류	Tx 없이 실행	Tx 가 있으면 안 되는 작업
`NESTED`	Savepoint 기반 중첩	새 Tx 처럼 시작	DB/Manager 가 중첩 Tx 를 실제 지원할 때만

가장 많이 쓰는 기준 일반 업무는 `REQUIRED`, 순수 조회는 `readOnly=true`, 주 트랜잭션과 독립적으로 반드시 남겨야 하는 DB 기록은 별도 Service 에 `REQUIRES\_NEW`를 적용하는 방식이 기본입니다.

6.3 트랜잭션을 분리하는 방법 - `REQUIRES\_NEW`

주 업무가 롤백되더라도 감사/시도 이력처럼 독립적으로 확정해야 하는 DB 작업은 별도 Spring Bean 의 메서드로 분리하고 `Propagation.REQUIRES\_NEW` 를 적용합니다. 같은 클래스 내부에서 `this.method()` 로 호출하면 Proxy 경계를 우회할 수 있으므로 별도 Service 호출을 기본으로 합니다.

```
@Service
public class MemberAuditService extends MemberBaseService {
    @Cpftx(
        id = "MBR_AUDIT_WRITE",
        name = "회원 감사 기록",
        ownerDomain = "MBR",
        transactionManager = "memberTransactionManager",
        propagation = Propagation.REQUIRES_NEW)
    public void writeAudit(MemberAudit audit) {
        auditRepository.insert(audit);
    }
}

@Service
public class MemberService extends MemberBaseService {
    private final MemberAuditService auditService;

    @Cpftx(
        id = "MBR_MEMBER_CHANGE",
        name = "회원 변경",
        ownerDomain = "MBR",
        transactionManager = "memberTransactionManager")
    public void changeMember(ChangeRequest request) {
        memberRepository.update(...);    // 주 트랜잭션
        auditService.writeAudit(...);    // 별도 REQUIRES_NEW 트랜잭션
        if (request.forceFailure()) {
            throw new CpfBusinessException("주 거래 실패");
        }
    }
}
```

실행 결과	주 트랜잭션	REQUIRES_NEW	의미
전체 성공	COMMIT	COMMIT	업무와 독립 기록 모두 반영
감사 성공 후 주 업무 실패	ROLLBACK	COMMIT	독립 기록은 유지
감사 자체 실패	호출 예외 정책에 따라 Rollback 가능	ROLLBACK	감사 실패를 무시할지/주 거래도 실패할지 업무 정책 필요
주 Tx 없음에서 감사 호출	해당 없음	새 Tx COMMIT/ROLLBACK	독립 서비스만 단독 실행 가능

실제 Education 예제 `cpf-education/.../EducationTransactionEducationAuditService.writeAuditRequiresNew()` 가 `@Cpftx(... propagation = Propagation.REQUIRES\_NEW)` 를 사용합니다. `EducationCrudEducationService.runSeparatedTransactionEducation()` 은 감사 기록 뒤 주 트랜잭션을 실패시키는 교육 흐름을 제공합니다.

6.4 조회 전용과 TransactionManager 선택

```
@CpfTx(
  id = "MBR_MEMBER_DETAIL",
  name = "회원 상세 조회",
  ownerDomain = "MBR",
  transactionManager = "memberTransactionManager",
  readOnly = true,
  timeoutSeconds = 3)
public MemberResponse getMember(String memberId) {
  return repository.findById(memberId)
    .map(this::toResponse)
    .orElseThrow(() -> new CpfNotFoundException("회원 없음"));
}
```

- `readOnly=true`는 조회 의도를 명확히 하고 Provider/DB 최적화에 사용할 수 있습니다. 조회 메서드에서 변경 SQL 을 섞지 않습니다.
- PlatformTransactionManager 가 하나면 `transactionManager`를 생략할 수 있습니다. 여러 Manager 가 있으면 CPF Interceptor 가 명시를 요구하며 모호한 선택을 허용하지 않습니다.
- `timeoutSeconds`는 0 이상일 때 Spring TransactionDefinition timeout 으로 적용됩니다. 외부 HTTP Timeout 과는 별개입니다.

6.5 Rollback-only 검증 실행

DB 에 실제 변경을 남기지 않고 SQL/Mapping/제약을 검증해야 할 때는 `CpfDataOperations.inRollbackOnlyTransaction()` 또는 `CpfJdbcOperations.inRollbackOnlyTransaction()`을 사용할 수 있습니다. 이 Helper 는 callback 을 실행한 뒤 enclosing transaction 을 항상 rollback-only 로 표시합니다.

```
validationResult = dataOperations.inRollbackOnlyTransaction(ops -> {
  ops.insert("MBR.member.insert", candidate);
  return ops.selectOne("MBR.member.detail", candidate.id());
});
// 검증은 수행하지만 DB 반영은 남기지 않음
```

6.6 Local Transaction 과 원격 호출의 경계

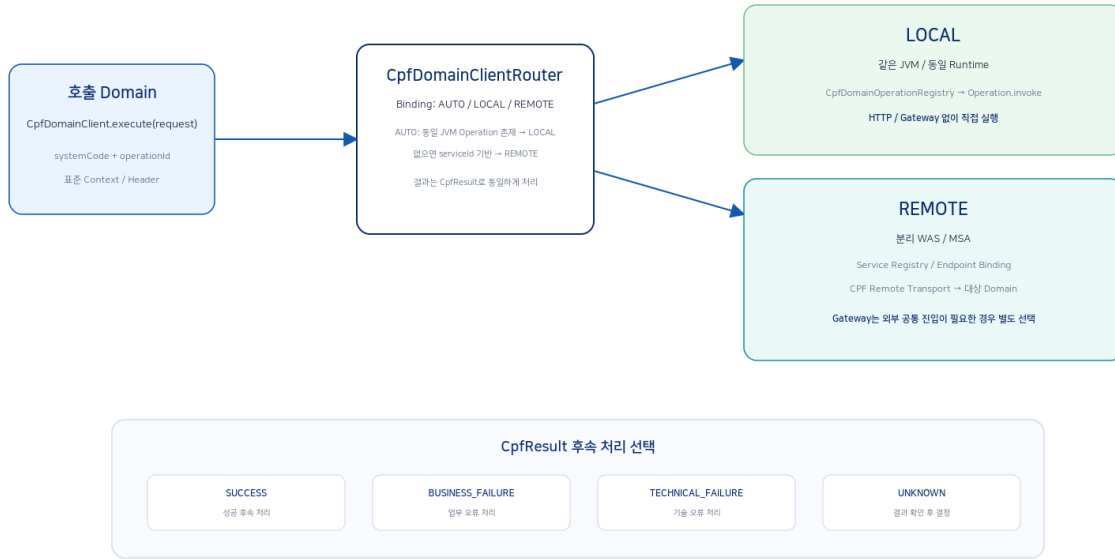
Local DB Transaction 은 Remote Domain/외부 시스템의 원자성을 보장하지 않습니다. 원격 호출을 DB Transaction 안에 오래 붙잡아 두지 말고, 강한 원자성이 필요한 경우에는 XA/JTA, Outbox, Saga, TCC 중 일관성 모델을 선택합니다.

상황	권장 모델	핵심 이유
단일 DB/Resource	Local `@CpfTx`	가장 단순하고 명확한 원자성
DB + DB 강한 원자성	XA/JTA	두 XA Resource 의 prepare/commit/rollback
DB + JMS 강한 원자성	XA/JTA 또는 Outbox 선택	Resource 특성과 운영 복잡도 판단
DB + Kafka/RabbitMQ/Event	Outbox 우선	DB 변경과 Outbox insert 를 같은 Local Tx 로 묶음
MSA A→B→C 장기 흐름	Saga	서비스별 Local Tx + 보상/상태 전이
잔액/한도/재고 Hold	TCC	Try/Confirm/Cancel 명시
외부 처리 결과 불명	UNKNOWN + Reconcile	Blind Retry 로 중복 Side Effect 를 만들지 않음

7. Domain 간 거래

도메인간 거래 호출 흐름

업무 코드는 동일 Typed Contract를 사용하고 Runtime Binding이 LOCAL / REMOTE를 선택



JTA를 사용하지 않는 본산 환경에서는 UNKNOWN을 일반 실패와 합치지 않습니다.

같은 Business Contract 를 유지하면서 LOCAL 또는 REMOTE 로 실행되는 Domain Call

방식	공개 API/설정	특징	선택 기준
같은 Domain 내부	직접 Service 호출	동일 Domain 내부 구현	별도 Domain 계약 불필요
Typed Domain Call	`CpfDomainClient` / `CpfDomainClientRouter.invoke()`	LOCAL/REMOTE 동일 `CpfResult`	다른 CPF Domain 호출
AUTO	`CpfDomainBindingMode.AUTO`	Local Operation 이 있으면 Local, 아니면 Remote	배포 형태 변경 가능
LOCAL	`CpfDomainBindingMode.LOCAL`	같은 JVM Operation 강제	통합 배포
REMOTE	`CpfDomainBindingMode.REMOTE`	Remote Transport 강제	분리 WAS/MSA

```
CpfResult<MemberResponse> result = domainClientRouter.invoke(
    "MBR", "OMBR0001", request, MemberResponse.class);
```

```
return result.fold(
    success -> handleSuccess(success),
    business -> handleBusinessFailure(business),
    technical -> handleTechnicalFailure(technical),
    unknown -> reconcileUnknown(unknown));
```

결과	확정 여부	재시도 판단	후속 처리
SUCCESS	성공 확정	불필요	정상 후속 처리
BUSINESS_FAILURE	업무 실패 확정	보통 재시도 안 함	업무 오류 응답/분기
TECHNICAL_FAILURE	기술 실패 확정	정책/역등성 확인 후	Retry 또는 실패 처리
UNKNOWN	상대 처리 여부 불명	Blind Retry 금지	operation/transaction 식별자로 조회·대사 후 결정

8. 외부 Service Call

외부 시스템이나 공통 Service Registry 기반 호출은 `CpfServiceCaller`와 `CpfServiceCallCommand` 계열을 사용합니다. Domain Call 과 외부 Service Call 을 같은 것으로 취급하지 않습니다.

옵션	API	역할	주의
대상	`serviceId`	Registry/Binding 대상	환경별 주소를 업무 코드에 하드코딩하지 않음
Method	`httpMethod()`	HTTP Method	업무 API 계약과 일치
Path	`requestPath()`	요청 경로	Base URL 과 분리
Timeout	`timeoutMillis()`	원격 호출 제한 시간	DB Tx timeout 과 별개
Retry	`retryCount()`	재시도 횟수	역등/Side Effect 안전성 확인 후
Header	`header(k,v)`	표준/업무 Header	Secret/PII 직접 로그 금지
Attribute	`attribute(k,v)`	호출 정책 부가 정보	공개 정책 키만 사용

9. 오류·검증·동시성·중복 요청

필요	CPF 기능/계약	핵심 기준	실패 시
입력 형식 검증	Jakarta Validation + Base Helper	Controller 경계에서 빠르게 실패	표준 Validation 오류
업무 검증	Service 업무 규칙	형식 검증과 분리	CpfBusinessException 계열
Not Found	`CpfNotFoundException`	조회 대상 없음	표준 오류 Mapping
중복 요청	`@CpfIdempotent`	업무 Key/결과 저장 범위 정의	동일 요청 재실행 방지
동시 수정	Optimistic Version	Version 충돌 감지	Conflict 처리
강한 잠금	`CpfLockMode.PESSIMISTIC_*`	실제 DB 불변조건 보호 필요 시	Lock timeout/실패 처리
분산 Lock	Lock Provider	DB Lock 으로 해결되지 않는 다중 Instance 자원	작업 소유권·timeout·fencing 정책
Cache	Caffeine/Redis/Valkey	TTL·무효화·다중 Instance 고려	원본 DB 를 정합성 기준으로 유지

10. 주요 개발 Annotation

Annotation	역할	적용 위치	주의
`@CpfController`	CPF Web Controller 등록/공통 연결	Business Controller	Spring MVC 계약을 불필요하게 숨기지 않음
`@CpfService`	업무 Service 등록/공통 연결	Business Service	업무 로직 소유
`@CpfRepository`	필요할 때 Repository 표준 표시	Repository/Adapter	모든 Repository 에 강제하지 않음
`@CpfTx`	Local Persistence Transaction	Service/DAO Method	원격 원자성을 보장하지 않음
`@CpfIdempotent`	중복 요청 제어	Side Effect 업무	Idempotency Key/결과 정책 필요
`@CpfRetry`	재시도 정책	Integration 경계	역등성이 확인된 호출에만
`@CpfTimeout`	호출 제한 시간	Integration 경계	DB Tx timeout 과 구분
`@CpfClient`	Integration Client 표준 연결	외부 Client	Provider 구현과 업무 계약 분리
`@CpfMessageListener`	메시지 Consumer 등록	비동기 Consumer	중복/Retry/DLQ 고려
`@CpfAudit`	감사 이벤트 연결	감사 대상 작업	민감정보 원문 금지

Annotation	역할	적용 위치	주의
`@CpfPermission`	권한 요구	보호 API/Service	Security 정책과 연계
`@CpfApprovalRequired`	승인 필요 작업	위험 운영/업무 조치	사유·승인·감사 이력 필요
`@CpfLogging` / `@CpfPerformance`	추가 로깅/성능 계측	필요한 경계	모든 메서드에 기계적으로 사용하지 않음

11. End-to-End 예제 - DB 조회 거래

1. Request DTO 에 Path/Query/Body 와 Jakarta Validation 을 정의합니다.
2. Controller 에서 필수 업무 키를 검증하고 Service 만 호출합니다.
3. Service 에 `@CpfTx(readOnly=true)` 를 선언하고 조회 원자성/timeout 을 정합니다.
4. Repository 에서 단건/Page/Slice/Cursor 중 요구에 맞는 계약을 선택합니다.
5. JDBC/MyBatis/JPA Provider 가 SQL/Mapper/Entity 실행을 담당합니다.
6. Not Found/Validation/Conflict 를 표준 예외 계약으로 반환합니다.
7. Integration Test 에서 공식 DB Vendor 또는 해당 Provider fixture 로 실제 Query 를 검증합니다.
8. transactionId 로 Log/Trace/ADM 조회가 연결되는지 확인합니다.

```
@CpfTx(
    id = "MBR_MEMBER_SEARCH",
    name = "회원 목록 조회",
    ownerDomain = "MBR",
    readOnly = true,
    timeoutSeconds = 3)
public CpfPage<MemberSummary> search(MemberSearchCriteria criteria, Integer page, Integer size) {
    CpfPageRequest request = CpfPageRequest.of(page, size);
    return repository.page(criteria, request, List.of(CpfSort.asc("memberId")));
}
```

12. End-to-End 예제 - 주 거래와 독립 감사 트랜잭션

9. 주 Service 는 `REQUIRED` 기본 트랜잭션에서 업무 데이터를 변경합니다.
10. 독립 기록은 별도 Bean 의 `REQUIRES\_NEW` 메서드로 호출합니다.
11. 독립 기록 성공 후 주 업무가 실패하면 주 DB 변경은 Rollback 되고 독립 기록은 Commit 된 상태로 남습니다.
12. 독립 기록 실패를 주 거래 실패로 연결할지 별도로 삼길지는 업무 정책으로 명시합니다.
13. 프로세스/DB 자체 장애처럼 commit 여부를 확정할 수 없는 경우에는 단순히 성공으로 가정하지 않고 운영 조회/복구 절차를 둡니다.

Education 확인 경로 `cpf-education/src/main/java/com/cpf/education/data/transaction/application/
 EducationTransactionEducationAuditService.java`, `cpf-education/src/main/java/com/cpf/education/web/crud/application/
 EducationCrudEducationService.java` 의 분리 트랜잭션 예제를 먼저 확인합니다.

13. 테스트 및 완료 확인

검증 단계	확인 내용	권장 위치
Unit	Validation, 업무 분기, CpfResult 4 상태	각 Domain `src/test`
Persistence Integration	CRUD/Page/Slice/Cursor/Lock/Tx Rollback	Provider/Generated Domain Test
Transaction	REQUIRED/REQUIRES_NEW/readOnly/timeout/rollback-only	`cpf-education/.../data/transaction/**`

검증 단계	확인 내용	권장 위치
Domain Call	AUTO/LOCAL/REMOTE, 결과 4 상태	Integration/Generated Domain Test
External Call	Timeout/Retry/UNKNOWN/Idempotency	Education resilience/servicecall Test
Security	401/403/Permission/Session	Education security Test
Runtime	Health/Trace/transactionId/ADM 조회	실제 Runtime Profile

14. 실제 코드 참조 위치

확인 대상	Source 위치
Controller Base	`cpf-starters/web/src/main/java/com/cpf/web/api/CpfBaseController.java`
Service Base	`cpf-starters/base/runtime/src/main/java/com/cpf/foundation/api/CpfBaseService.java`
DAO Base	`cpf-starters/data/persistence/src/main/java/com/cpf/data/persistence/api/CpfBaseDao.java`
Transaction Annotation	`cpf-starters/data/persistence/src/main/java/com/cpf/data/persistence/api/annotation/CpfTx.java`
Repository API	`cpf-starters/data/persistence/src/main/java/com/cpf/data/persistence/api/**`
Paging API	`cpf-starters/data/persistence/src/main/java/com/cpf/data/persistence/api/page/**`
Domain Call	`cpf-starters/integration/src/main/java/com/cpf/integration/api/domaincall/**`
분산 결과	`cpf-core/src/main/java/com/cpf/core/api/result/CpfResult.java`
Service Call	`cpf-starters/integration/src/main/java/com/cpf/integration/api/servicecall/**`
Generated Sample	`cpf-member/**`, `cpf-external/**`
실행형 학습	`cpf-education/**`, `cpf-docs/development/EDU_GUIDE.md`

15. 개발 완료 점검

- Controller 가 HTTP/입력 경계만 담당하고 업무 규칙이 Service 에 있는가?
- Framework Base 와 Domain Base 의 공통 메서드를 다시 구현하지 않았는가?
- DB 조회 방식(Page/Slice/Cursor)과 Repository 계약을 요구에 맞게 선택했는가?
- `@CpfTx` 의 readOnly/전파/Manager/timeout 을 업무 원자성에 맞게 정했는가?
- 독립 트랜잭션이 필요하면 별도 Bean + `REQUIRES\_NEW` 로 실제 Proxy 경계를 만들었는가?
- Remote Domain/External Call 을 Local DB Tx 와 같은 원자성으로 가정하지 않았는가?
- UNKNOWN 결과를 Blind Retry 하지 않고 조회/대사 식별자를 보존하는가?
- Idempotency, Version/Lock, Cache 무효화가 필요한 Side Effect 를 검토했는가?
- 실제 Provider/DB/Runtime Test 와 transactionId 기반 운영 확인까지 완료했는가?